EDUCATION / PHYSICAL COMPUTING

ESP32 Education Editor

Complete Manual

Editor v0.6 | Firmware v0.1.8

Target board: ESP32-WROOM-32 / ESP32 Dev Module
Project: davinichi / ESP32 Education Editor
English Edition

ESP32 Education Editor Manual

This complete English manual covers ESP32 Education Editor v0.6 / Firmware v0.1.8.

Included Manuals

- **User Guide** - setup, connection, editor operation, saving, ESP-NOW, and hardware notes
- **Block Reference** - all 49 public blocks in 9 categories

Version Scope

Item	Version / Scope
Editor	v0.6
Firmware	v0.1.8
Target board	ESP32-WROOM-32 / ESP32 Dev Module
Extension categories	9
Public blocks	49

> The editor can switch between Japanese and English. Projects use common internal extension IDs and opcodes so that the same project can be opened in either language.

ESP32 Education Editor User Guide

> This guide explains the basic setup and use of ESP32 Education Editor for educational physical-computing activities.

1. What is ESP32 Education Editor?

ESP32 Education Editor is an educational block-programming environment for controlling an ESP32-WROOM-32 from a web browser. Firmware v0.1.8 runs on the ESP32, while the editor sends commands over USB / Web Serial.

Editor v0.6 **Firmware v0.1.8** **ESP32-WROOM-32**

- Public Editor: https://davinichi.github.io/
- Firmware Installer: https://davinichi.github.io/firmware/

System Overview



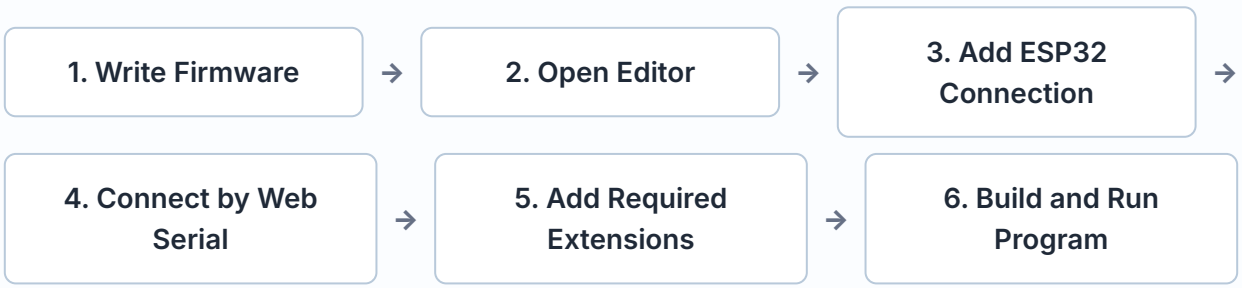
2. What You Need

- ESP32-WROOM-32
- A USB cable that supports data communication
- A Web Serial compatible browser such as Chrome or Edge
- Internet access when loading the public Editor or Firmware Installer
- Sensors, LEDs, OLEDs, servo motors, or other components required by the activity
- Appropriate resistors and power supplies for the circuit

> **Voltage warning:** The HC-SR04 ECHO output is approximately 5 V. Do not connect it directly to an ESP32 input. Reduce the signal to 3.3 V or below.

3. First-Time Setup

Basic Workflow



3.1 Write the Firmware

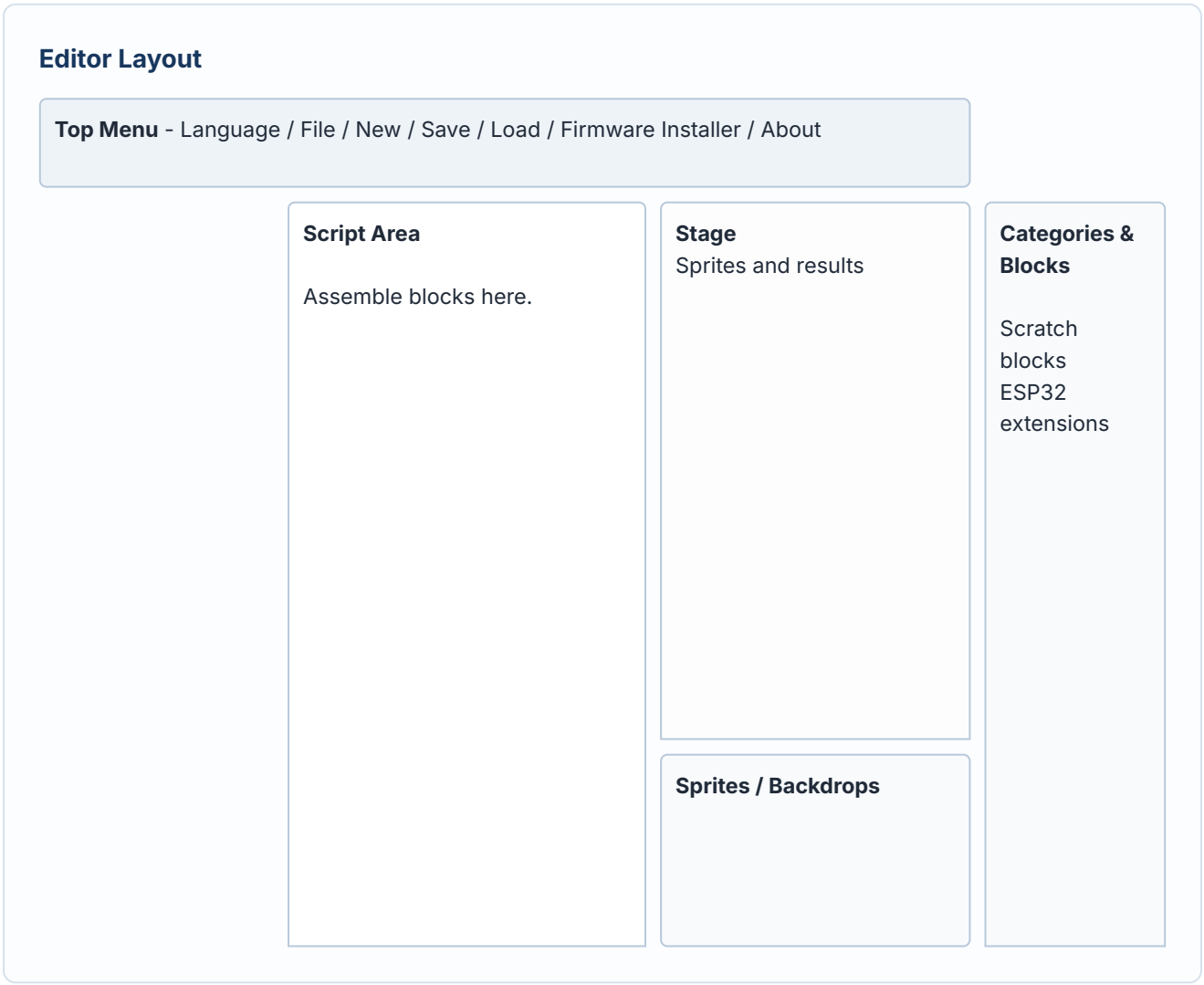
1. Connect the ESP32-WROOM-32 to the PC by USB.
2. Open the Firmware Installer at https://davinichi.github.io/firmware/.
3. Choose the option to connect to the ESP32 and write Firmware v0.1.8.
4. Select the ESP32 USB serial port.
5. Follow the on-screen instructions to complete the installation.

You do not need to prepare Arduino IDE or additional Arduino libraries on the user PC.

3.2 Open the Editor

Open https://davinichi.github.io/ in Chrome or Edge. Use the language menu at the top of the editor to switch between Japanese and English.

4. Editor Layout



- **Top menu:** language, file operations, new project, save/load, Firmware Installer, and About
- **Categories / blocks:** standard Scratch categories plus the added ESP32 extensions
- **Script area:** where blocks are assembled into programs
- **Stage:** displays sprites and program results
- **Sprites / backdrops:** add and select sprites or backgrounds

5. Connect to the ESP32

1. Click the Extensions button at the lower-left of the editor.
2. Add **ESP32 Connection**.
3. Run the **Connect to ESP32** block.
4. Select the ESP32 serial port from the browser chooser.
5. If needed, run **Refresh ESP32 status** and check the status, MAC address, and channel.

> If Arduino IDE Serial Monitor or another application is using the same serial port, disconnect or close it before connecting from the browser.

6. Add Extension Categories

Category	Main Use	ESP32 Connection
ESP32 Connection	Connection, status, MAC address, channel	Required
ESP32 GPIO	Digital I/O and PWM	Required
ESP32 DHT	DHT11 / DHT22 temperature and humidity	Required
ESP32 OLED	SSD1306 OLED display	Required
ESP32 ESP-NOW	Text communication between ESP32 devices	Required
ESP32 Servo	Servo motor control	Required
ESP32 Ultrasonic	HC-SR04 distance measurement	Required
ESP32 Environment Index	10 environmental-index calculations	Not required
Data Processing	CSV and text processing	Not required

See the **Block Reference** for details of each block.

7. Building Basic Programs

7.1 LED Example

1. Add ESP32 Connection and connect to the ESP32.
2. Add ESP32 GPIO.
3. Run **set GPIO 23 to output**.
4. Run **write HIGH to GPIO 23** to turn the LED on.
5. Run **write LOW to GPIO 23** to turn the LED off.

> Use an appropriate current-limiting resistor with an LED and verify the wiring before powering the circuit.

7.2 Sensor Value Display Example

1. Initialize the DHT sensor.
2. Read DHT temperature and humidity.

3. Initialize the OLED.
4. Set the cursor position and text size.
5. Convert the measured values to text and display them on the OLED.

8. Running Programs

Combine standard Scratch event and control blocks with the ESP32 extension blocks. Typical blocks include:

- when green flag clicked
- forever
- wait ( ) seconds
- if ( ) then
- repeat ( )
- variables and operators

> The current editor uses a live-control model. The PC, browser, and USB connection are required while the program is running.

9. Saving and Loading Projects

Use the **File** menu at the top of the editor.

- **New:** start a new project
- **Load from your computer:** open a saved project
- **Save to your computer:** save the current project locally

The Japanese and English interfaces share the same internal extension IDs and opcodes, so the same project is designed to open in either language.

10. Using ESP-NOW

ESP-NOW allows ESP32 devices to communicate directly.

- School Wi-Fi access point: not required
- School LAN: not required
- Internet during ESP-NOW communication: not required
- ESP32 channel: set the sender and receiver to the same value
- Blank destination MAC: broadcast
- Specified destination MAC: unicast

Internet access is still needed when the public Editor or Firmware Installer is loaded from the web.

11. Hardware Safety and Usage Notes

- The supported target is ESP32-WROOM-32.
- Check the board specification for GPIO numbering, voltage limits, and input-only pins.
- Use a current-limiting resistor with LEDs.
- Check the power requirements of the servo motor used.
- Reduce the HC-SR04 ECHO signal to 3.3 V or below.
- Disconnect power or USB when changing wiring when appropriate.
- Avoid assigning the same GPIO to multiple functions at the same time.

12. Troubleshooting Checklist

1. Is the board an ESP32-WROOM-32?
2. Has Firmware v0.1.8 been written to the board?
3. Are you using Chrome or Edge?
4. Does the USB cable support data communication?
5. Is another application using the serial port?
6. Did you add the extension required by the program?
7. Are the sensor type, GPIO numbers, wiring, and power correct?

13. Related Documents

- ESP32 Education Editor Block Reference
- Getting Started (project document)
- FAQ / Troubleshooting (project document)
- Firmware Guide (project document)
- Lesson / Educational Use Guide (project document)
- Backup / Restore Guide (project document)

At the time of this English edition, some related project documents may still be available only in Japanese.

14. Project Positioning

ESP32 Education Editor is an independent project based on the open-source Scratch Editor published by the Scratch Foundation. It is not an official product of the Scratch Foundation and does not imply endorsement, recommendation, or sponsorship by the Scratch Foundation.

ESP32 Education Editor Block Reference

> This reference describes the 49 public blocks shown in ESP32 Education Editor v0.6, organized by category.

> The block illustrations in this English edition are explanatory diagrams based on the current English labels, default values, category colors, and block types. Actual appearance may vary slightly by browser and screen width.

Block Shapes

- **Command block:** performs an action.
- **Reporter block:** returns a number or text value.
- **Boolean block:** returns true or false for use in conditions.

Contents

ESP32 Connection
ESP32 GPIO
ESP32 DHT
ESP32 OLED
ESP32 ESP-NOW
ESP32 Servo
ESP32 Ultrasonic
ESP32 Environment Index
Data Processing

ESP32 Connection

Connect to an ESP32 by USB / Web Serial and check connection state, MAC address, and Wi-Fi channel.

Requirements: Requires an ESP32 running Firmware v0.1.8. Add this category first.

1. Connect to ESP32

Connect to ESP32

Type	Command block
Function	Connects the browser to the ESP32 using Web Serial.
Inputs	None
Output	None
Notes	Running the block opens the browser serial-port chooser. Close Arduino IDE Serial Monitor or any other app using the same port first.
Example	Run this block before using GPIO or sensor blocks.

2. Disconnect from ESP32

Disconnect from ESP32

Type	Command block
Function	Closes the Web Serial connection to the ESP32.
Inputs	None
Output	None
Notes	Use before another application needs the serial port or before unplugging the USB cable.
Example	Explicitly close the ESP32 connection at the end of a session.

3. ESP32 connected?

ESP32 connected?

Type	Boolean block
Function	Returns whether the editor is currently connected to an ESP32.
Inputs	None
Output	True when connected; false when disconnected
Notes	Useful when program behavior should depend on connection state.
Example	Use in a condition such as "if ESP32 connected? then".

4. Refresh ESP32 status

Refresh ESP32 status

Type	Command block
Function	Requests the latest status information from the ESP32.
Inputs	None
Output	None
Notes	Run when you want to refresh status, MAC address, and Wi-Fi channel information.
Example	Run after connecting, then read the status or MAC reporter blocks.

5. ESP32 status

ESP32 status

Type	Reporter block
Function	Returns the last status string received from the ESP32.
Inputs	None
Output	Status string; may be blank before data is received
Notes	Check after connection or after refreshing ESP32 status.
Example	Display the status on the stage to confirm the connection.

6. ESP32 MAC address

ESP32 MAC address

Type	Reporter block
Function	Returns the MAC address of the connected ESP32.
Inputs	None
Output	MAC address string
Notes	Useful when identifying a destination for ESP-NOW communication.
Example	Check the MAC addresses of two ESP32 boards before setting up unicast communication.

7. ESP32 Wi-Fi channel

ESP32 Wi-Fi channel

Type	Reporter block
Function	Returns the Wi-Fi channel currently used by the ESP32.
Inputs	None
Output	Channel number
Notes	ESP-NOW devices that communicate with each other must use the same channel.
Example	Compare both devices when ESP-NOW reception fails.

ESP32 GPIO

Control GPIO modes, digital output, digital input, and PWM.
Requirements: Requires an ESP32 connection. Check the GPIO number and external circuit before use.

8. set GPIO [PIN] to [MODE]

set GPIO [PIN] to [MODE]

Type	Command block
Function	Sets the input/output mode of a selected GPIO pin.
Inputs	PIN: GPIO number / MODE: output, input, input pull-up, or input pull-down
Output	None
Notes	Set the appropriate mode before digital input or output operations.
Example	Set GPIO 23 to output before driving an LED.

9. write [STATE] to GPIO [PIN]

write [STATE] to GPIO [PIN]

Type	Command block
Function	Outputs HIGH or LOW on the selected GPIO pin.
Inputs	PIN: GPIO number / STATE: HIGH or LOW
Output	None
Notes	Use after configuring the pin as an output.
Example	Set GPIO 23 HIGH to turn an LED on and LOW to turn it off.

10. set GPIO [PIN] PWM to [PERCENT] %

set GPIO [PIN] PWM to [PERCENT] %

Type	Command block
Function	Outputs PWM from 0 to 100 percent on the selected GPIO.
Inputs	PIN: PWM-capable GPIO / PERCENT: 0 to 100
Output	None
Notes	The value is rounded to an integer and limited to 0-100. Firmware v0.1.8 uses 5 kHz, 8-bit PWM.
Example	Set an LED brightness to about 50 percent.

11. stop PWM on GPIO [PIN]

stop PWM on GPIO [PIN]

Type	Command block
Function	Stops PWM output on the selected GPIO.
Inputs	PIN: PWM-capable GPIO
Output	None
Notes	PWM can be started again later.
Example	Stop LED dimming controlled by PWM.

12. digital input GPIO [PIN]

digital input GPIO [PIN]

Type	Reporter block
Function	Reads the digital input value from the selected GPIO.
Inputs	PIN: GPIO number
Output	0 or 1
Notes	The reporter requests a read and returns the most recently received value. It may return 0 before data is received.
Example	Turn on an LED when a switch input becomes 1.

ESP32 DHT

Read temperature and relative humidity from DHT11 / DHT22 sensors.
Requirements: Requires an ESP32 connection and sensor initialization with the correct type and GPIO.

13. initialize [TYPE] on GPIO [PIN]

initialize [TYPE] on GPIO [PIN]

Type	Command block
Function	Sets the DHT sensor type and GPIO pin in the firmware.
Inputs	TYPE: DHT11 or DHT22 / PIN: GPIO number
Output	None
Notes	Run once before reading temperature or humidity.
Example	For a DHT11 on GPIO 25, select DHT11 and enter 25.

14. DHT temperature (deg C)

DHT temperature (deg C)

Type	Reporter block
Function	Reads the DHT sensor and returns temperature in degrees Celsius.
Inputs	None
Output	Temperature in deg C; may return 0 on read failure
Notes	Running the reporter sends a DHT read request to the ESP32.
Example	Show a warning when the temperature is 30 deg C or higher.

15. DHT humidity (%)

DHT humidity (%)

Type	Reporter block
Function	Reads the DHT sensor and returns relative humidity.
Inputs	None
Output	Relative humidity in percent; may return 0 on read failure
Notes	Running the reporter sends a DHT read request to the ESP32.
Example	Display humidity on the OLED.

16. DHT status

DHT status

Type	Reporter block
Function	Returns the most recent status string received for the DHT sensor.
Inputs	None
Output	Status string
Notes	Use as a diagnostic aid for initialization and sensor reads.
Example	Display the status while checking wiring.

ESP32 OLED

Initialize and use an SSD1306 128x64 OLED, including text display, partial clearing, and status checks.

Requirements: Requires an ESP32 connection and wiring compatible with the firmware configuration.

17. initialize OLED

initialize OLED

Type	Command block
Function	Initializes the SSD1306 OLED.
Inputs	None
Output	None
Notes	Run before using other OLED blocks.
Example	Run first when the green flag is clicked.

18. clear OLED screen

clear OLED screen

Type	Command block
Function	Clears the entire OLED display to black.
Inputs	None
Output	None
Notes	Useful before redrawing the whole screen.
Example	Clear before displaying a new measurement screen.

19. fill OLED X [X] Y [Y] W [W] H [H] black

fill OLED X [X] Y [Y] W [W] H [H] black

Type	Command block
Function	Fills only the selected rectangular area with black.
Inputs	X: 0-127 / Y: 0-63 / W: width / H: height
Output	None
Notes	Coordinates and dimensions are constrained to the 128x64 display area.
Example	Erase only the numeric value area before updating it.

20. set OLED cursor X [X] Y [Y]

set OLED cursor X [X] Y [Y]

Type	Command block
Function	Sets the starting position for the next text output.
Inputs	X: 0-127 / Y: 0-63
Output	None
Notes	The upper-left corner is X=0, Y=0.
Example	Set X=0, Y=16 to start a second text line area.

21. set OLED text size [SIZE]

set OLED text size [SIZE]

Type	Command block
Function	Sets the text size used for OLED output.
Inputs	SIZE: 1-8
Output	None
Notes	The value is rounded to an integer and constrained to 1-8.
Example	Use size 2 for a larger heading.

22. show [TEXT] on OLED

show [TEXT] on OLED

Type	Command block
Function	Displays a string at the current OLED cursor position.
Inputs	TEXT: string to display
Output	None
Notes	Newline characters are replaced by spaces.
Example	Display temperature and humidity converted to text.

23. OLED display test

OLED display test

Type	Command block
Function	Runs the OLED test routine provided by the firmware.
Inputs	None
Output	None
Notes	Use to confirm wiring and initialization.
Example	Run as a pre-class hardware check.

24. OLED status

OLED status

Type	Reporter block
Function	Returns the most recently received OLED status.
Inputs	None
Output	READY, ERROR, or a blank string
Notes	Useful for checking the initialization result.
Example	If the status is not READY, check wiring and power.

25. OLED last response

OLED last response

Type	Reporter block
Function	Returns the most recent response to an OLED command.
Inputs	None
Output	Last ACK/response string
Notes	Use to inspect responses from initialization, clear, cursor, or text operations.
Example	Display during debugging to confirm command acknowledgement.

ESP32 ESP-NOW

Send and receive text directly between ESP32 devices without using the school Wi-Fi access point or LAN.
Requirements: Both devices require Firmware v0.1.8. Use the same channel and specify a MAC address when needed.

26. set ESP-NOW channel to [CHANNEL]

set ESP-NOW channel to [CHANNEL]

Type	Command block
Function	Sets the Wi-Fi channel used by ESP-NOW to a value from 1 to 13.
Inputs	CHANNEL: 1-13
Output	None
Notes	All communicating ESP32 devices must use the same channel. After restart, the channel returns to 1.
Example	Set both sender and receiver to channel 6.

27. ESP-NOW channel

ESP-NOW channel

Type	Reporter block
Function	Returns the current ESP-NOW channel.
Inputs	None
Output	Channel number; blank when not connected
Notes	Use after connection to verify the active channel.
Example	Compare both devices if communication fails.

28. send [MESSAGE] to destination MAC [MAC]

send [MESSAGE] to destination MAC [MAC]

Type	Command block
Function	Sends a text string using ESP-NOW.
Inputs	MAC: destination MAC address or blank / MESSAGE: text to send
Output	None
Notes	A blank MAC sends a broadcast; specifying a MAC sends unicast. Newlines are replaced by spaces.
Example	Leave MAC blank and send "Hello" to all listening devices.

29. ESP-NOW received data

ESP-NOW received data

Type	Reporter block
Function	Returns the body of the most recently received ESP-NOW message.
Inputs	None
Output	Received string
Notes	Reading this value clears the "new data" flag.
Example	When new data is available, read this reporter.

30. new ESP-NOW data received?

new ESP-NOW data received?

Type	Boolean block
Function	Returns whether unread ESP-NOW data is available.
Inputs	None
Output	True when new unread data exists; otherwise false
Notes	Reading the received-data reporter resets this to false.
Example	Check inside a forever loop for new messages.

31. ESP-NOW send result

ESP-NOW send result

Type	Reporter block
Function	Returns the result of the most recent ESP-NOW transmission.
Inputs	None
Output	OK, FAIL, or an error string
Notes	Use to check destination MAC format and peer configuration.
Example	After sending, confirm the result is OK.

ESP32 Servo

Attach and control a small servo motor such as the SG90 by angle.
Requirements: Requires an ESP32 connection. Follow the power and wiring specifications of the servo used.

32. attach servo to GPIO [PIN]

attach servo to GPIO [PIN]

Type	Command block
Function	Assigns the GPIO used for servo control.
Inputs	PIN: servo GPIO
Output	None
Notes	Run before setting an angle.
Example	Attach a servo to GPIO 13.

33. set servo angle to [ANGLE] degrees

set servo angle to [ANGLE] degrees

Type	Command block
Function	Sets the angle of the attached servo.
Inputs	ANGLE: 0-180 degrees
Output	None
Notes	The value is rounded to an integer and constrained to 0-180.
Example	Move to 90 degrees for a center position.

34. detach servo

detach servo

Type	Command block
Function	Stops servo output and detaches it from the GPIO.
Inputs	None
Output	None
Notes	Use when servo control is finished.
Example	Detach after a sequence of movements is complete.

ESP32 Ultrasonic

Measure distance in centimeters using an HC-SR04 ultrasonic sensor.
Requirements: Requires an ESP32 connection. Reduce the HC-SR04 ECHO voltage to 3.3 V or below before the ESP32 input.

35. attach ultrasonic sensor TRIG [TRIG] ECHO [ECHO]

attach ultrasonic sensor TRIG [TRIG] ECHO [ECHO]

Type	Command block
Function	Sets the GPIO pins used for the HC-SR04 TRIG and ECHO signals.
Inputs	TRIG: output GPIO / ECHO: input GPIO
Output	None
Notes	Run before measuring distance. Do not connect the approximately 5 V ECHO signal directly to an ESP32 input. Reduce it to 3.3 V or below.
Example	Use GPIO 26 for TRIG and GPIO 34 for ECHO.

36. ultrasonic distance (cm)

ultrasonic distance (cm)

Type	Reporter block
Function	Measures distance with the HC-SR04 and returns centimeters.
Inputs	None
Output	Distance in cm; -1 if measurement fails
Notes	Check sensor orientation, target surface, wiring, and voltage levels.
Example	Trigger an LED or buzzer when distance is below 20 cm.

ESP32 Environment Index

Calculate 10 environmental indices from temperature and humidity. Calculations are performed in the editor, so an ESP32 connection is not required.
Requirements: Provide temperature and humidity values, for example from the DHT blocks.

Available Environmental Indices

Display Name	Internal Value	Summary
Discomfort Index (DI)	`DI`	Index representing perceived thermal discomfort
Heat Index	`HEAT_INDEX`	Apparent heat estimated from temperature and humidity (deg C)
Humidex	`HUMIDEX`	Index of muggy heat based on temperature and humidity
Dew Point (Td)	`DEW_POINT`	Temperature at which water vapor begins to condense (deg C)
Absolute Humidity (AH)	`ABS_HUM`	Mass of water vapor per cubic meter of air (g/m3)
Wet-Bulb Temperature (Tw)	`WET_BULB`	Approximate wet-bulb temperature (deg C)
VPD	`VPD`	Vapor pressure deficit: saturation minus actual vapor pressure (kPa)
Vapor Pressure (VP)	`VP`	Actual water vapor pressure (kPa)
Temperature-Humidity Index (THI)	`THI`	Temperature-humidity index using temperature and dew point
Estimated WBGT (simple)	`WBGT`	Simple estimate derived from wet-bulb temperature and air temperature (deg C)

37. calculate [INDEX] from temperature [TEMP] deg C and humidity [HUM] %

calculate [INDEX] from temperature [TEMP] deg C and humidity [HUM] %

Type	Reporter block
Function	Calculates the selected environmental index from temperature and relative humidity in the editor.
Inputs	TEMP: temperature in deg C / HUM: relative humidity in percent / INDEX: index type
Output	Calculated value for the selected index
Notes	Humidity is constrained to 0-100 percent. Units depend on the selected index.
Example	Calculate an estimated WBGT value from DHT temperature and humidity.

Data Processing

Parse CSV data, format numbers, and extract or join text. Processing is performed in the editor.
Requirements: No ESP32 or firmware is required.

38. split CSV data [CSV]

split CSV data [CSV]

Type	Command block
Function	Splits one line of CSV text at commas and stores the parsed fields internally.
Inputs	CSV: text to parse
Output	None
Notes	Commas inside quoted fields are handled. Parsing fails if quotation marks are not properly closed.
Example	Split "25.6,60,28.3" into three fields.

39. CSV item [INDEX] as text

CSV item [INDEX] as text

Type	Reporter block
Function	Returns the selected field from the most recently parsed CSV as text.
Inputs	INDEX: 1-based field number
Output	String; blank when out of range
Notes	Run the split CSV block first.
Example	Read the second field "60" as text.

40. CSV item [INDEX] as number

CSV item [INDEX] as number

Type	Reporter block
Function	Converts the selected CSV field to a number and returns it.
Inputs	INDEX: 1-based field number
Output	Number; 0 if conversion fails
Notes	Use when the field is needed for calculations.
Example	Read the first field "25.6" as a numeric value.

41. CSV item count

CSV item count

Type	Reporter block
Function	Returns the number of fields in the most recently parsed CSV.
Inputs	None
Output	Number of fields
Notes	Returns 0 before parsing.
Example	Check whether a received line has the expected number of fields.

42. CSV split succeeded?

CSV split succeeded?

Type	Boolean block
Function	Returns whether the most recent CSV parse succeeded.
Inputs	None
Output	True on success; false on failure
Notes	Malformed quoted CSV returns false.
Example	Use the parsed fields only when this block returns true.

43. format number [VALUE] to [DIGITS] decimal places

format number [VALUE] to [DIGITS] decimal places

Type	Reporter block
Function	Converts a number to a string with the specified number of decimal places.
Inputs	VALUE: number / DIGITS: 0-10
Output	Formatted string; blank if VALUE is not numeric
Notes	DIGITS is converted to an integer and constrained to 0-10.
Example	Format 25.376 to 2 places and get "25.38".

44. left [COUNT] characters of [TEXT]

left [COUNT] characters of [TEXT]

Type	Reporter block
Function	Returns the specified number of characters from the start of a string.
Inputs	TEXT: source string / COUNT: character count
Output	Extracted string
Notes	Works by characters, including non-ASCII text.
Example	Return "ABC" from "ABC123" with COUNT=3.

45. right [COUNT] characters of [TEXT]

right [COUNT] characters of [TEXT]

Type	Reporter block
Function	Returns the specified number of characters from the end of a string.
Inputs	TEXT: source string / COUNT: character count
Output	Extracted string
Notes	COUNT=0 returns an empty string.
Example	Return "123" from "ABC123" with COUNT=3.

46. length of [TEXT]

length of [TEXT]

Type	Reporter block
Function	Returns the number of characters in a string.
Inputs	TEXT: source string
Output	Character count
Notes	Counts Unicode code points.
Example	"ABC123" returns 6.

47. from character [START] of [TEXT] to end

from character [START] of [TEXT] to end

Type	Reporter block
Function	Returns the substring starting at the selected 1-based character position.
Inputs	TEXT: source string / START: starting position beginning at 1
Output	Extracted string
Notes	START is treated as at least 1.
Example	From "2026/08/01", START=6 returns "08/01".

48. [COUNT] characters from character [START] of [TEXT]

[COUNT] characters from character [START] of [TEXT]

Type	Reporter block
Function	Returns a substring of the requested length starting at the selected position.
Inputs	TEXT: source string / START: 1-based start / COUNT: character count
Output	Extracted string
Notes	COUNT is treated as 0 or greater.
Example	From "ABCDEFG", START=3 and COUNT=2 returns "CD".

49. join [A] and [B] with [SEP]

join [A] and [B] with [SEP]

Type	Reporter block
Function	Joins two values with the selected separator.
Inputs	A: first value / B: second value / SEP: separator
Output	Joined string
Notes	Numeric values are converted to text before joining.
Example	Join 25.6 and 60 with a comma to get "25.6,60".

Hidden Compatibility Block

For compatibility with existing projects, the internal block `WBGT [WBGT] warning level` remains in the project source but is not shown in the new block palette. It is therefore not included in the 49 public blocks described in this reference.

Related Documents

- ESP32 Education Editor User Guide
- Getting Started (project document)
- FAQ / Troubleshooting (project document)
- Firmware Guide (project document)